

Silent Guidance - Multiple Prior Executions

User selects EC for execution:

EC: Experience Driver -> Intent Type -> Response Type -> Casual Proxy

Silent Guidance queries IML:

Found: 5 prior executions of semantically identical EC

- Execution #1: 18 months ago
- Execution #2: 14 months ago
- Execution #3: 8 months ago
- Execution #4: 3 months ago
- Execution #5: 1 month ago

The wall you hit: "Do I show all 5? Just the most recent? What's the decision logic that makes this INSTRUCTIVE, not just informative?"

The Core Design Question

This isn't just "what to display" — it's "**what intelligence does the user need to make the right decision?**"

Let me explore multiple approaches:

Approach 1: Naive (Show Everything)

Display all 5 executions chronologically with metadata:

⚠ Silent Guidance: This EC has been executed 5 times previously

Execution #1 18 months ago	
Owner: Sarah Chen (Product Team)	
Initiative: Introduced order status emails	
Elasticity: 0.45 (Moderate - fragile improvement)	

Disposition: Redo (improvement reverted after 3mo)	
Execution #2 14 months ago	
Owner: Marcus Williams (Operations)	
Initiative: Added +1 warehouse shift	
Elasticity: 0.68 (High - durable improvement)	
Disposition: Standardize (worked well, now protocol)	
Execution #3 8 months ago	
Owner: Sarah Chen (Product Team)	
Initiative: Real-time tracking page	
Elasticity: 0.52 (Moderate - improvement held)	
Disposition: Control (monitor for drift)	
Execution #4 3 months ago	
Owner: Alex Kumar (Logistics)	
Initiative: Carrier performance penalties	
Elasticity: 0.31 (Low - minimal impact)	
Disposition: Redesign (wrong approach)	
Execution #5 1 month ago	
Owner: Jordan Lee (Engineering)	
Initiative: Automated delay notifications	
Elasticity: [Pending - in monitoring window]	
Disposition: Active	

Pros:

- Complete transparency
- User sees full history
- Can learn from patterns

Cons:

- Cognitive overload (too much data)
- User must manually synthesize meaning
- **No guidance — just information dump**

Verdict: This is what most systems would do. It's not instructive enough.

Approach 2: Intelligent Synthesis (Extract Pattern)

Instead of showing raw history, **synthesize what the history means**:

⚠️ Silent Guidance: This issue has been addressed 5 times before

📊 Pattern Analysis:

🔍 What We've Learned:

- This is a RECURRING issue (5 attempts in 18 months)
- 2 approaches worked (Elasticity >0.6), 3 failed or fragile
- Success pattern: Operational changes (warehouse shift, tracking)
- Failure pattern: Messaging/notification solutions (band-aids)

✅ What Worked Best:

- "Added +1 warehouse shift" (14 mo ago)
 - Elasticity: 0.68 (High durability)
 - Still in effect today
 - Cost: \$180K/year, impact: 40% complaint reduction

⚠️ What Didn't Work:

- "Carrier performance penalties" (3 mo ago)
 - Elasticity: 0.31 (Low impact)
 - Root cause: penalties didn't change carrier behavior

🎯 Strategic Insight:

- This appears to be a SYSTEMIC issue, not a process issue.
The fact that it resurfaces despite fixes suggests:
- Demand seasonality outpacing capacity
 - OR structural constraint in fulfillment model

💡 Recommendation:

- Before initiating another fix:
1. Review why prior successful solution (#2) isn't holding
 2. Consider whether this needs "Optimize" stream (capacity model) rather than "Fix" stream (tactical improvement)
 3. If proceeding with Fix, favor operational interventions over communication solutions (proven higher elasticity)

🔗 [View detailed history →](#)

Pros:

- Synthesizes learning (user doesn't have to)
- Provides strategic context (recurring = systemic)
- Offers actionable recommendation
- Pattern recognition (what worked, what didn't)

Cons:

- Requires sophisticated analysis logic
- Could miss nuance user would catch
- Prescriptive (might override user judgment)

Verdict: This is much more instructive. But can we go further?

Approach 3: Decision-Tree Guidance (Interactive Intelligence)

Don't just show history — **guide the decision** interactively:

⚠ Silent Guidance: This EC has been executed 5 times before

? Before proceeding, let's ensure we're not repeating history:

DECISION CHECKPOINT #1
Question: Is this the SAME underlying cause as previous attempts?
Context: Prior executions targeted:
• 40% → Process bottlenecks (warehouse capacity)
• 40% → Communication gaps (customer notifications)
• 20% → Carrier performance
👉 What's your diagnosis this time?
○ Same as before: [Warehouse capacity]
○ Same as before: [Communication gaps]
○ Same as before: [Carrier performance]
○ NEW root cause: [_____]

[User selects: "Same as before: Warehouse capacity"]

DECISION CHECKPOINT #2

⚠️ We've addressed warehouse capacity before:

"Added +1 warehouse shift" (14 months ago)

→ Worked well initially (Elasticity 0.68)

→ Still operational today

→ But issue has resurfaced anyway

😞 This suggests:

- Demand has grown beyond added capacity
- OR seasonal spike exceeding permanent solution
- OR new constraint emerged downstream

👉 Before adding MORE capacity:

Do you want to:

- Investigate why prior solution no longer holds
- Proceed with capacity addition anyway
- Consider different approach (dynamic staffing?)

[User selects: "Investigate why prior solution no longer holds"]

GUIDED INVESTIGATION

✅ Smart choice. Let's understand what changed:

📊 Data since last fix (14 months ago):

- Order volume: +65% (explains capacity pressure)
- Delivery complaints: Back to pre-fix levels
- Peak complaint days: Weekends (new pattern)

💡 Hypothesis: Weekend demand spike overwhelming weekday capacity additions

🎯 Suggested EC Reframe:

Instead of: Fix | Delivery Speed | Late
Processing | Checkout

Consider: Optimize | Delivery Speed | Weekend
Capacity Model | Fulfillment

This routes to "Optimize" stream (capacity planning) rather than "Fix" stream (tactical intervention)

👉 Would you like to:

○ Reframe as Optimize EC (recommended)	
○ Continue with original Fix EC	
○ Cancel and investigate further	

Pros:

- **Deeply instructive** (teaches user to think systemically)
- Prevents blind repetition
- Guides toward root cause
- Can redirect to better EC formulation
- Interactive (user maintains agency while getting guidance)

Cons:

- More complex to build (decision tree logic)
- Could feel patronizing if user is expert
- Longer time to execution (friction)

Verdict: This is the most instructive, but needs careful UX.

Approach 4: Stratified Display (Tiered Intelligence)

Show different levels based on **what the history reveals**:

Tier 1: Single Prior Execution

 This EC was executed once before (6 months ago)

Quick Summary:

- Initiative: [Name]
- Elasticity: 0.72 (High - worked well)
- Status: Standardized and still in effect

 Insight: Prior solution is holding. This recurrence suggests:

- New root cause emerged, OR
- Increased volume overwhelming prior fix

→ View full details | Proceed with execution

Lightweight — just enough context to be helpful.

Tier 2: Multiple Executions, Different Outcomes

⚠ This EC has been executed 4 times with MIXED results

Pattern Recognition:

✓ 2 successes (Elasticity >0.6)

⚠ 1 fragile (Elasticity 0.4-0.6, later reverted)

✗ 1 failure (Elasticity <0.4)

What distinguished success from failure:

- Successes targeted: [Operational capacity]
- Failures targeted: [Communication band-aids]

🎯 Recommendation: Focus on operational interventions

→ Show detailed comparison | Proceed with caution | Reframe EC

Medium detail — highlights pattern without overwhelming.

Tier 3: Multiple Executions, All Failed

🚨 ALERT: This EC has been attempted 3 times — ALL failed

History of Failure:

Execution #1 (12 mo ago): Elasticity 0.28 (Failed)

Execution #2 (8 mo ago): Elasticity 0.33 (Failed)

Execution #3 (4 mo ago): Elasticity 0.19 (Failed)

📊 Pattern: Each attempt had LOWER impact than previous

→ Suggests issue is not addressable via this EC formulation

⚠ STRONG RECOMMENDATION: Do not proceed as-is

Consider:

1. Cross-Issue Learning: Is this symptom of broader problem?
2. EC Reframing: Should this be Optimize vs. Fix?
3. Root Cause Review: Are we solving the wrong problem?

→ Explore alternatives | Override and proceed (not recommended)

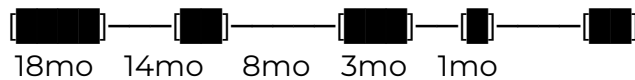
High urgency — strong guidance to prevent waste.

Tier 4: Recurring Pattern (Sign of Systemic Issue)

● SYSTEMIC PATTERN DETECTED

This EC has been executed 5 times over 18 months

Timeline:



Each fix provided temporary relief, but issue recurs

🔍 This is NOT a "fix once" problem — it's STRUCTURAL

What this means:

- Tactical fixes will continue to degrade
- Root cause is likely:
 - Demand pattern (seasonal, growth-driven)
 - Capacity model inadequacy
 - Systemic constraint upstream/downstream

🎯 RECOMMENDATION: Escalate to strategic review
This belongs in "Optimize" or "Innovate" stream,
not "Fix" stream

Options:

1. Create CSLI investigation (Cross-Issue Learning)
2. Reframe as Optimize EC (capacity/process redesign)
3. Escalate to leadership (strategic decision needed)
4. [Not recommended] Proceed with another tactical fix

→ Launch CSLI | Reframe EC | Escalate | Proceed anyway

Maximum guidance — system recognizes systemic pattern and redirects.

My Recommended Approach: Adaptive Tiered Display

Don't pick ONE approach. Use different approaches based on WHAT THE HISTORY REVEALS:

```
function determineSilentGuidanceStrategy(priorExecutions) {

  // Tier 1: Single prior execution
  if (priorExecutions.length === 1) {
    return {
      strategy: "SUMMARY",
      display: "Lightweight summary with quick context",
      guidance: "Minimal — show what happened, let user decide"
    };
  }

  // Tier 4: Systemic pattern (5+ attempts, recurring)
  if (priorExecutions.length >= 5) {
    const avgTimeBetween = calculateAvgTimeBetween(priorExecutions);
    if (avgTimeBetween < 6_MONTHS) {
      return {
        strategy: "SYSTEMIC_ALERT",
        display: "Strong warning with pattern visualization",
        guidance: "Redirect to CSLI or Optimize/Innovate stream"
      };
    }
  }

  // Tier 3: Multiple failures
  if (priorExecutions.length >= 3) {
    const allFailed = priorExecutions.every(e => e.elasticity < 0.4);
    if (allFailed) {
      return {
        strategy: "FAILURE_PATTERN",
        display: "Alert with strong recommendation to reframe",
        guidance: "Block execution unless override with justification"
      };
    }
  }

  // Tier 2: Mixed results (default for 2-4 executions)
  return {
    strategy: "PATTERN_SYNTHESIS",
```

```
display: "Success vs. failure comparison",  
guidance: "Highlight what worked, suggest approach alignment"  
};  
}
```

The Decision Logic Components

For EACH tier, compute these intelligences:

1. Recurrence Velocity

recurrenceVelocity = avgTimeBetweenExecutions

```
if (recurrenceVelocity < 3_MONTHS) {  
  signal = "ACUTE_RECURRING"; // Fires constantly  
} else if (recurrenceVelocity < 6_MONTHS) {  
  signal = "CHRONIC_RECURRING"; // Persistent issue  
} else {  
  signal = "PERIODIC"; // Occasional resurfacing  
}
```

Use this to determine urgency of systemic investigation.

2. Elasticity Trajectory

elasticityTrend = linearRegression(priorExecutions.map(e => e.elasticity))

```
if (elasticityTrend.slope < -0.1) {  
  signal = "DEGRADING"; // Each attempt works less  
} else if (elasticityTrend.slope > 0.1) {  
  signal = "IMPROVING"; // Getting better at solving it  
} else {  
  signal = "STABLE"; // Consistent impact  
}
```

Use this to predict whether current attempt likely to work.

3. Approach Diversity

uniqueApproaches = new Set(priorExecutions.map(e => e.initiativeType))

```

if (uniqueApproaches.size === 1) {
  signal = "MONOLITHIC"; // Trying same thing repeatedly
} else if (uniqueApproaches.size >= 3) {
  signal = "EXPLORATORY"; // Tried many approaches
}

```

Use this to recommend trying different angle vs. refining same approach.

4. Success Pattern Recognition

```

successes = priorExecutions.filter(e => e.elasticity > 0.6)
failures = priorExecutions.filter(e => e.elasticity < 0.4)

```

```

successPatterns = extractCommonalities(successes) // e.g., "operational changes"
failurePatterns = extractCommonalities(failures) // e.g., "communication fixes"

```

```

if (successPatterns.length > 0) {
  guidance = `Successes shared: ${successPatterns.join(', ')}`
  recommendation = "Align your approach with proven patterns"
}

```

Use this to guide user toward high-probability approaches.

5. Current Context Comparison

```

// Compare CURRENT situation to PRIOR executions
currentContext = {
  volumeGrowth: +65%, // From SEU temporal data
  seasonality: "weekend_spike_new",
  priorFixesStillActive: true
}

```

```

// If context has CHANGED significantly, prior outcomes may not apply
if (contextHasShifted(currentContext, priorExecutions)) {
  guidance = "Context has changed significantly since prior attempts"
  recommendation = "Prior patterns may not hold — proceed with fresh analysis"
}

```

Use this to determine if history is still relevant.

The UX Flow (Bringing It Together)

Step 1: User Selects EC

EC: Fix | Delivery Speed | Late Processing | Checkout Stage

Step 2: Silent Guidance Activates

[System queries IML]

- Found 5 prior executions
- Analyze: Recurrence velocity = CHRONIC (avg 3.6 months)
- Analyze: Elasticity trajectory = DEGRADING (slope -0.08)
- Analyze: Success pattern = "Operational capacity additions"
- Analyze: Context shift = SIGNIFICANT (volume +65%)
- Determine tier: SYSTEMIC_ALERT

Step 3: Display Adaptive Guidance

 SYSTEMIC PATTERN DETECTED

[Visualization of 5 attempts over timeline]

This issue recurs every 3-4 months despite interventions.

Pattern Analysis:

- Successes: Capacity additions (but degrading over time)
- Failures: Communication/notification band-aids
- Context: Volume +65% since last fix overwhelmed solution

 This is a STRATEGIC issue, not tactical

Recommendations (ranked):

1.  Launch Cross-Issue Learning Investigation
 - Explore if this is symptom of broader capacity model issue
2. Reframe as "Optimize" EC (capacity planning, not quick fix)
3. Escalate to leadership (may require capital investment)
4.  NOT RECOMMENDED: Another tactical "Fix"
 - History shows diminishing returns on tactical approaches

Choose action: [1] [2] [3] [Override and proceed anyway]

Step 4: User Makes Informed Decision

Either:

- **Accepts guidance** → Redirects to CSLI or Optimize stream
 - **Overrides** → Must provide justification (logged for learning)
-

The Intelligence Hierarchy

Your Silent Guidance should operate at three levels:

Level 1: INFORM

- └ Show what happened before
- └ Present data neutrally
- └ User interprets

Level 2: ADVISE

- └ Synthesize patterns
- └ Recommend approach
- └ User decides

Level 3: REDIRECT

- └ Recognize systemic patterns
- └ Block ineffective paths
- └ Guide to better formulation

Use Level 1 for simple cases, Level 3 for obvious waste prevention.

The Specific Answer to Your Question

"Do you show all examples timestamped?"

Answer: Depends on what the history teaches.

My recommendation:

1. **Always compute** full analysis (recurrence velocity, elasticity trajectory, success patterns, context shifts)
 2. **Adaptively display** based on tier:
 - 1 prior execution → Lightweight summary
 - 2-4 mixed results → Pattern synthesis (success vs. failure)
 - 3+ all failures → Strong warning with reframe suggestion
 - 5+ recurring → Systemic alert with strategic redirection
 3. **Always offer** "View detailed history" link for users who want full data
 4. **Never just dump** all 5 executions without synthesis — that's lazy design
-

The Implementation

// Silent Guidance Decision Engine

```
function generateGuidance(ec, priorExecutions) {
```

```
  // Step 1: Compute intelligences
```

```
  const intelligence = {  
    recurrenceVelocity: computeRecurrenceVelocity(priorExecutions),  
    elasticityTrend: computeElasticityTrend(priorExecutions),  
    successPatterns: extractSuccessPatterns(priorExecutions),  
    failurePatterns: extractFailurePatterns(priorExecutions),  
    contextShift: analyzeContextShift(ec, priorExecutions)  
  };
```

```
  // Step 2: Determine tier
```

```
  const tier = determineTier(priorExecutions, intelligence);
```

```
  // Step 3: Generate guidance based on tier
```

```
  switch(tier) {
```

```
    case "SUMMARY":
```

```
      return generateSummaryGuidance(priorExecutions[0]);
```

```
    case "PATTERN_SYNTHESIS":
```

```
      return generatePatternGuidance(intelligence);
```

```
    case "FAILURE_PATTERN":
```

```
      return generateFailureWarning(intelligence);
```

```
    case "SYSTEMIC_ALERT":
```

```
      return generateSystemicAlert(intelligence, priorExecutions);
```

```
}  
}
```

The Key Insight

Silent Guidance isn't about showing data — it's about teaching decision-making.

When you show 5 prior executions, you're asking the user to:

1. Spot patterns themselves
2. Synthesize meaning themselves
3. Decide implications themselves

That's not guidance. That's a quiz.

Real guidance:

- Spots patterns FOR them
 - Synthesizes meaning FOR them
 - Recommends action BASED ON synthesis
 - But preserves user agency (they can override)
-

My Specific Recommendation

Build Silent Guidance with **four adaptive display modes** triggered by the decision logic above:

1. **Summary Mode** (1 prior execution)
2. **Pattern Mode** (2-4 mixed executions)
3. **Warning Mode** (3+ failures or degrading elasticity)
4. **Redirect Mode** (5+ recurring or systemic pattern detected)

In EVERY mode:

- Provide "View all history" option (for detail seekers)
- Explain WHY you're showing what you're showing
- Give clear recommendation
- Allow override with justification requirement

This makes Silent Guidance genuinely intelligent, not just a database query.
